electronics

MDPI

Article

A High-Performance FPGA-Based RoCE v2 RDMA Packet Parser and Generator

Zezheng Sun [1,2], Zhichuan Guo [1,2,*], Jiandong Ma [1,2] and Yipeng Pan [1,2]

[1] National Network New Media Engineering Research Center, Institute of Acoustics, Chinese Academy of Sciences, No. 21, North Fourth Ring Road, Haidian District, Beijing 100190, China; sunzz@dsp.ac.cn (Z.S.); majd@dsp.ac.cn (J.M.); panyp@dsp.ac.cn (Y.P.)

[2] School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, No. 19(A), Yuquan Road, Shijingshan District, Beijing 100049, China

* Correspondence: guozc@dsp.ac.cn

Abstract: RDMA (Remote Direct Memory Access) technology has been widely applied due to its high-throughput and low-latency characteristics compared with traditional networks. Implementing RDMA with an FPGA (Field-Programmable Gate Array) is a feasible solution. This paper proposes an implementation method for the ROCE v2 (Remote Direct Memory Access) protocol packet parser and generator based on an FPGA, capable of supporting various transaction packet types, such as RDMA READ, RDMA WRITE, and SEND, under the Reliable Connection service. The RDMA READ and RDMA WRITE performance of RDMA is close to 100 Gbps, which provides a feasible solution for the application of wide-area networks.

Keywords: FPGA; RDMA; RoCE v2; packet parser; packet generator

check for updates

Citation: Sun, Z.; Guo, Z.; Ma, J.; Pan, Y. A High-Performance FPGA-Based RoCE v2 RDMA Packet Parser and Generator. *Electronics* **2024**, *13*, 4107. https://doi.org/10.3390/electronics13204107

Academic Editors: Ke Xie, Shoue Chen and Xiaolong Wang

Received: 2 September 2024
Revised: 11 October 2024
Accepted: 16 October 2024
Published: 18 October 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

To face the computational needs of wide-area networks, big data, and other fields, hardware accelerator cards are being deployed on a larger scale. However, there is often a significant physical distance between these network nodes, requiring faster, low-latency data network transmission technologies.

To address these problems, RDMA (Remote Direct Memory Access) technology is extensively utilized. In RDMA technology, data are transmitted over the network between local and remote machines without the CPU's involvement. All the tasks related to transmission are offloaded to the RDMA network interface card. The data transfer between the user-space virtual memory and the RDMA NIC is conducted directly, without engaging the operating system's kernel, and without any additional data movement or copying. Therefore, compared with traditional TCP/IP and other methods that rely on the CPU and software protocol stacks, RDMA technology reduces the complexity associated with data transmission. Thus, it can provide high-throughput, low-latency communication.

The RDMA transmission system involves two parties: the client and the server. The data operation requests are initiated by the client and responded to by the server. As is shown in Figure 1, the connection-oriented reliable service includes two-sided "SEND/RECEIVE" operations and one-sided "RDMA READ" and "RDMA WRITE" operations. The two-sided operations require the participation of both hosts and are often used for the transmission of short control messages. The one-sided operations only require the participation of one host and are often used for actual data transfer. Requests such as "RDMA READ Request", "RDMA WRITE", and "SEND" are transmitted from the client to the server. After the server receives them, it generates corresponding "RDMA READ response", "Acknowledge (ACK)", and other response packets, which are sent from the server back to the client.

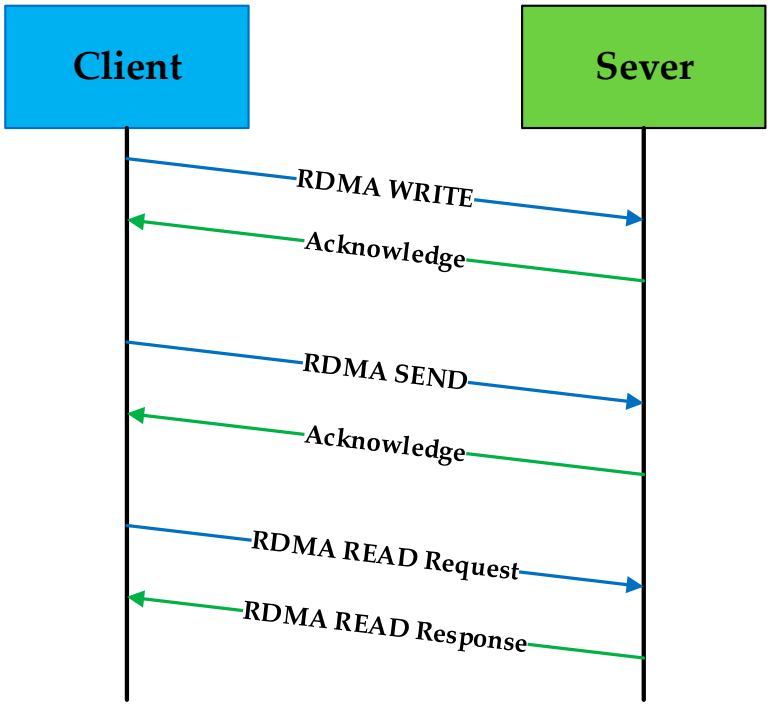

Figure 1. RDMA request and response.

Various transfer operations run on specific implementation protocols. The RoCE v2 (RDMA over Converged Ethernet version 2) [1] protocol, released by the InfiniBand Trade Association, builds RDMA on top of the Ethernet protocol's link layer and the IP protocol's network layer, encapsulating the RDMA Transport Protocol with the UDP protocol in the transport layer. As shown in Figure 2, the transport layer information of RDMA is carried by IB BTH+ (L4 Header), which marks the packet and indicates the specific RDMA transaction operation being executed. Compared with earlier InfiniBand technology, RoCE supports lossless Ethernet deployment without the need for proprietary networks, which allows traditional Ethernet traffic and RDMA traffic to coexist within the same network structure. It simplifies network management operations and eliminates the need for a separate RDMA architecture, greatly enhancing the usability and cost-effectiveness of RoCE v2. According to the white paper "ROCE in the Data Center" [2], published by Mellanox in 2014, it was not until the advent of RoCE that RDMA was widely adopted in Ethernet-based data centers, bringing low-latency, high-performance experiences to data center operations. Microsoft [3] and Oracle Cloud Infrastructure (OCI) [4] have utilized RoCE v2 technology to accelerate their high-performance computing, artificial intelligence, and database applications.

Figure 2. RoCE v2 packet.

An FPGA-based (Field-Programmable Gate Array) smart network interface card (smartnic) is a flexible and efficient networking device. It allows developers to program and configure hardware according to specific application needs, enabling reconfiguration to adapt to different network environments and protocol changes based on varying network protocols and application requirements [5]. Microsoft Azure [6] introduced the deployment of smart network cards on a large-scale cloud server in an article from 2018. Additionally, due to the hardware acceleration characteristics of FPGAs, smart network interface cards can achieve stronger processing capabilities and lower latency compared with traditional

CPUs, thereby accelerating the efficiency of network data processing. As a result, smart network cards can be designed as RDMA network interface cards to implement RoCE v2 protocol parsing. In the COnfigurable Network Protocol Accelerator [7] architecture proposed by Intel, two non-RoCE protocol RDMA operations, PUT and GET, are utilized. The Coyote [8] architecture proposed by Dario Korolija and others implements the functionality of RoCE to demonstrate its versatility. Arashloo M.T. [9] tested the performance of the RoCE v2 protocol on a 40 Gbps link in their work on transport layer protocol programming.

It can be seen that implementing the RoCE v2 protocol on an FPGA-based smart network card is a feasible and flexible solution. Our research focuses on the design of the RoCE v2 packet parser and generator to support RDMA traffic at 100 Gbps levels. The main contributions of this paper are as follows:

- We used Verilog (RTL) to implement the ROVE v2 high-speed parser and generator module on an FPGA. And it was tested to reach 100 Gbps, which is of great value for the application of RDMA.
- We propose a design architecture for the transmission and reception of RoCE v2 protocol packets, which supports the functional requirements of RDMA network cards to act as both clients and servers.
- We have built a software and hardware platform for RDMA transmission and reception tests.

2. System Design

We designed the system based on the RoCE v2 protocol and RDMA transport layer protocol to enable the network card to support both RDMA transmission and reception functions simultaneously. This design consists of two main pathways: the incoming pathway and the outgoing pathway. These two pathways work in concert to handle various RDMA transactions.

The incoming (RX) pathway receives data packets from the second-layer Ethernet, which are input through the AXI stream bus. After processing, the payload is output to the designated location in memory through the AXI memory mapped method.

The outgoing (TX) pathway extracts information from the Work Queue Element (WQE) to generate request data packets or the corresponding response data packets based on the received request data packets. The input for the outgoing pathway comes from the host memory carried by the AXI memory-mapped interface. Finally, all data packets are output to the Ethernet interface through the AXI stream bus.

We use a pipeline to design modules in the path, and the processing of each module is relatively independent. By improving and optimizing the design of the flowline, this design can operate at higher frequencies.

The overall block diagram structure of the design is shown in Figure 3 and is divided into the following key subsystems based on functionality.

RX engine: Responsible for the reception and processing of all data packets, as well as the writing of payloads. It generates information for the request manager subsystem to create response data packets for request packets. And it delivers information to the QP manager subsystem to indicate the completion of the corresponding work request for response packets.

Request manager: It manages two queues, one for caching the ACK information required for write requests and send requests and another for the read response information needed for read requests, passing it on to the TX engine.

QP manager: Responsible for managing the work queue and the completion queue, as well as storing queue configuration information.

TX engine: It receives the WQE from the work queue and response information from the request management, extracts the payload, completes the assembly of the data packet, and sends it out.

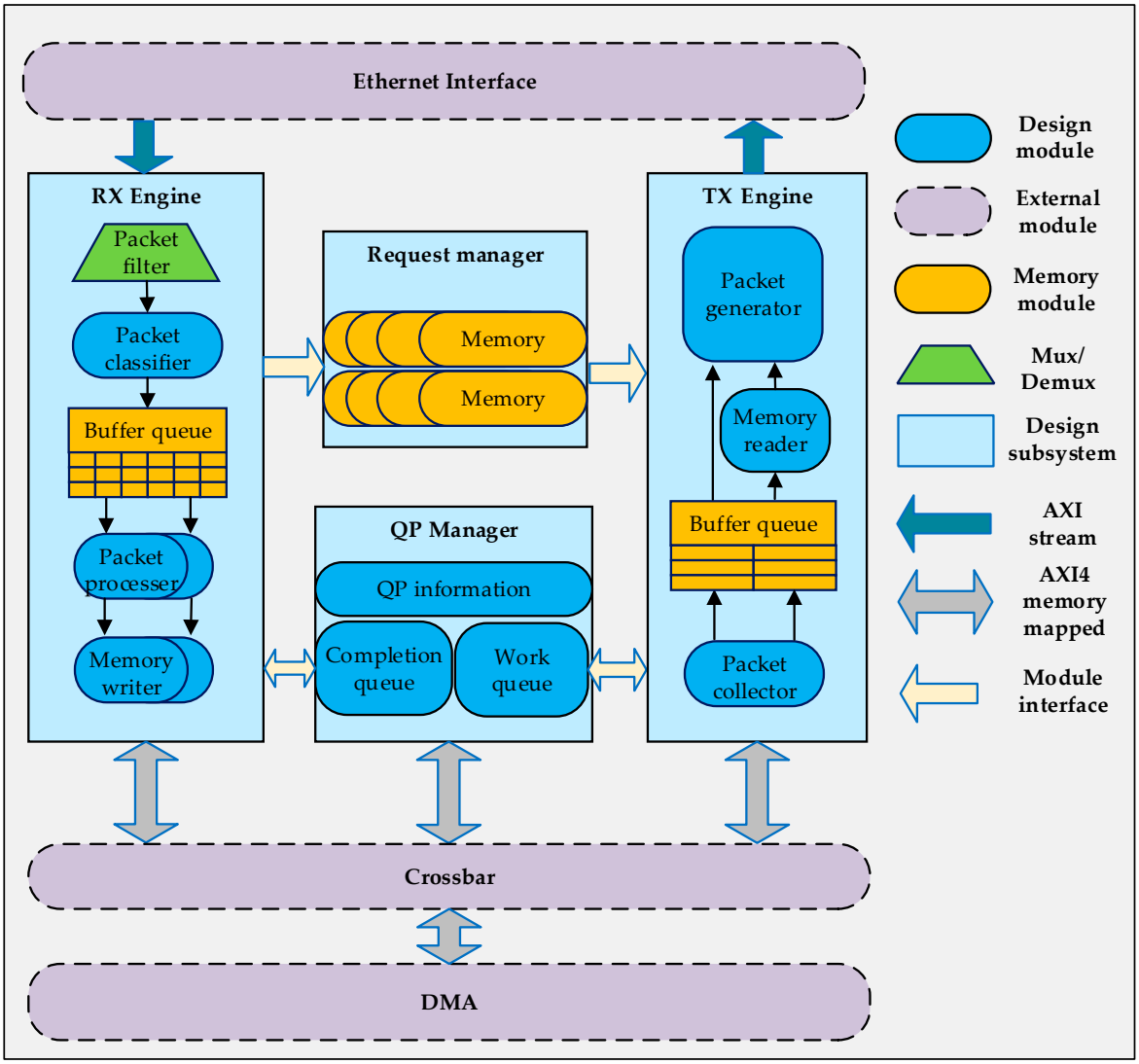

Figure 3. Block diagram.

In addition to the above subsystems, there are also necessary external modules. For example, the AXI memory-mapped interfaces of the RX engine, TX engine, and QP interaction subsystem are all connected to the AXI crossbar module, as well as the Ethernet and DMA modules.

2.1. RX Engine

As shown in Figure 4, the receiving engine inputs second-layer Ethernet packets carried by the AXI stream bus. It uses AXI memory-mapped interfaces to write the payload of the transmission into memory and outputs request information and completion information. During processing, packet classification is performed based on the type of packet, with packets for operations such as "WRITE", "READ request", and "SEND" being referred to as request packets, transmitted from the client to the server. Packets for operations such as "READ response" and "Acknowledge" are called response packets, sent from the server to the client. The following is a modular description according to the data packet processing procedure.

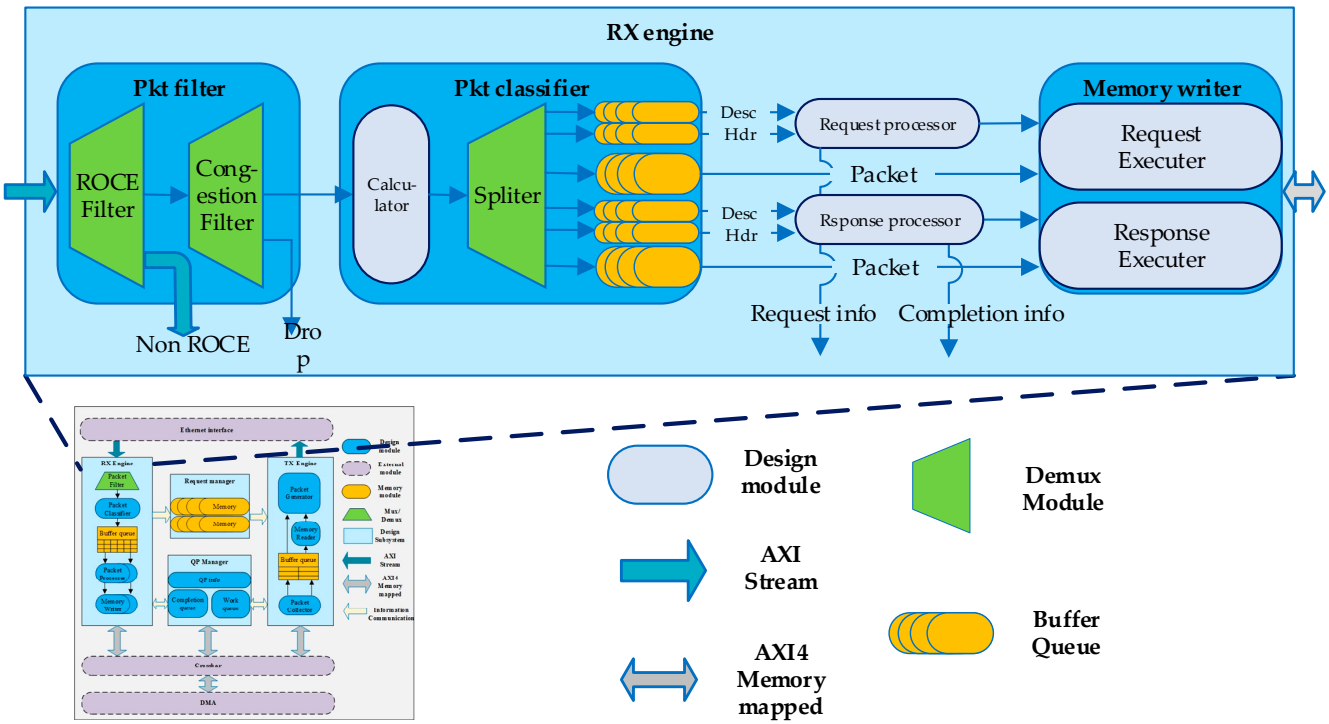

Figure 4. RX engine.

2.1.1. Packet Filter

Ethernet packets are transmitted into the packet filter via the AXI stream bus. Upon receiving the Ethernet packets, the packet filter proceeds to the RoCE filter module. This module reads the UDP header of each packet to determine if its port number matches the dedicated port number for the RoCE v2 protocol. If it does not, the packet is transmitted to an external interface by using a different set of AXI stream buses. If it does, the packet is forwarded to the congestion filtering module.

The congestion filtering module reads the status of the subsequent buffer queue, which is divided into two thresholds for discarding packets (i.e., the buffer is almost full, and the buffer is already full) to control the congestion of the buffer queue. Under the former conditions, it leverages the property that ACK packets can be merged and discards the ACK packets, thereby reducing the processing load. When the buffer queue is full, it discards all packets to ensure the normal operation of the subsequent modules.

2.1.2. Packet Classifier

The main function of the packet classifier is to perform the verification, classification, and splitting of data packets. It consists of two main components: the packet check calculator and the packet splitter.

The packet check calculator is responsible for calculating the checksum of the UDP/IP packet header and the ICRC (Invariant Cyclic Redundancy Check) value of RoCE v2. The calculation process of CRC utilizes LFSR (Linear Feedback Shift Register) technology. We refer to the CRC32 calculation rules specified in InfiniBand Architecture Specification Chapter 7, Section 8 [10] and the RoCE v2 [1] agreement.It solidifies the variable field value of the UDP/IP packet header to 1, using an initial seed of 0xFFFFFFFF and the specific generating polynomial. The polynomial is shown in Formula (1) below.

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^{8} + x^{7} + x^{5} + x^{4} + x^{2} + x + 1. \quad (1)$$

The calculator completes the checksum calculation and comparison to obtain the result and then inputs it into the splitter module. The splitter reads the operation code in the BTH

header of the data packet, classifies all data packets into request packets (SEND, RDMA WRITE, RDMA READ request, etc.) and response packets (Acknowledge and RDMA READ response) according to the operation, and then separates the packet header from each data packet. The separated content is stored in the corresponding cache queues, which are response packets, response packet headers, request packets, and request packet headers.

The RDMA packet must contain a header, while it does not necessarily contain payload, such as the Acknowledge packet. Moreover, in subsequent packet header processing, the processing result does not necessarily have to perform load writing. This decouples the packet header and packet load, requiring the maintenance of their correlation.

To maintain the correspondence between the packet header and the data packet, a descriptor is designed to store information such as the internal number of the data packet, cache location, data packet length, and check result. Two first-in-first-out descriptor queues are maintained, including the request descriptor queue and the response descriptor queue, which are transferred to the packet processor module.

All the above queues use independent queue pointers, which are moved based on the subsequent processing results to locate queue elements. The pointer cyclically increments within the design depth, achieving the effect of cyclically utilizing the cache queue. This independent queue setting avoids packet processing conflicts and achieves high-performance processing.

In our design, the depth of the queue is parameterizable. Greater depth means that more pending packets can be stored without pressuring the input Ethernet interface. In subsequent tests, we chose 512 (default value) as the queue depth for packet headers and descriptors. The packet queue depth matches accordingly, but the width is maintained at the AXI stream bus width of 512 bits.

2.1.3. Packet Processor

The key fields of an RoCE packet header are parsed and processed in the packet processor, thereby determining the operations to be implemented on the packet and the responses or completion information to be provided. The processor consists of two independent modules: the request packet processor module and the response packet processor module. They operate separately without interfering with each other. The processor receives packet headers and related descriptors, determines the method of accessing memory, and derives processing information.

This design allows our RX engine to serve as a reception module for both the server and the client, processing input data packets in parallel and allowing multiple transactions to run concurrently, thereby improving processing efficiency.

The request packet processor module becomes active when acting as a server. It processes and generates request information, determining the type and content of the response packet that should be sent in reply to the received request packet. The response packet processor module becomes active when acting as a client. It records the received response packets to obtain completion information of processing progress, indicating whether it has completed or executed the progress.

The packet processor is responsible for reading the corresponding packet header buffer queue and descriptor queue. After obtaining the packet header and descriptor, the module reads the RDMA configuration information and execution status information. Configuration information refers to the memory range, key, base address, and other transaction settings set after the RDMA connection is established. The execution status information records detailed information such as the processed packet sequence number, operation type, and message sequence number.

Additionally, the module parses the packet header layer by layer and extracts elements, including the Ethernet layer, IP layer, UDP layer, and RDMA transport layer. These extracted elements serve three main purposes.

The first step is to verify if they meet the requirements of the corresponding layer and RDMA protocol and to record errors if there are any differences. The second step is to

perform a classification based on the RDMA operation type for the execution of the appropriate processing procedure. The final step is to compare the configuration information and status information to ensure compliance with the RDMA protocol's regulations.

During the specific transaction processing, the results of previous checks constitute an important reference. If all checks pass, the subsequent execution module is instructed to complete the required memory access operations. The response packet processor outputs the confirmed completion information, and the request packet processor outputs the confirmed response information. Conversely, if errors are detected during the check process, the subsequent execution module is instructed to abandon the current packet. Then, it outputs non-acknowledge responses and incomplete information based on the specific nature of the error.

The packet processor sequentially processes each packet, with the primary overhead being the access to the packet header queue and the reading of configuration and status information. Even for the 256-byte RDMA packets with the minimum load, in the 512-bit AXI bus environment we operate in, the overhead of performing memory writes exceeds that of the processor. Hence, the processing time of the packet processor module does not impact the throughput.

2.1.4. Memory Writer

This module includes the executors for the request packet processor module and the response packet processor module, which are set independently and side by side, without affecting each other. It is responsible for receiving the processing results from the corresponding packet header processing modules and executing operations to write the packet payload into memory or discard it.

The module uses the AXI memory-mapped method to write to memory. Specifically, write address, write data, and write response channels are used while leaving vacant the read address and read data channels. Thus, the task of this module is to organize the actions of the write address and write data channels.

It receives the processing results of the data packet header to organize the actions. As mentioned earlier, the time cost of memory writing is greater than the processing speed of the preceding module. So, we design FIFOs at the input end to store the processing results of the preceding modules. The results include packet type, whether to write to memory, the memory address to be written, and the length of the packet. Based on the processing results, the module categorizes and processes different types of data packets to determine the initial address of the write address channel and the length of the write.

At the same time, this module receives the correspondence between the data packet and the header provided by the packet header processing module, which allows it to locate and retrieve the data packet from the cache queue. The payload of this data packet is then divided according to a fixed bit width, serving as the data to be written.

Once the slave end is ready, it executes the memory address and data writing. After receiving the write response, the transaction is completed. Then, the module continues to retrieve the next processing result from the FIFO and execute it in a loop.

2.2. Request Manager

When the RDMA network card is set to operate as a server, the request management module becomes active. This module takes the parsed request information from the RX engine and manages the generation of response packets such as acknowledgements and responses required to reply to incoming request packets. As is shown in Figure 5, the design content is as detailed below.

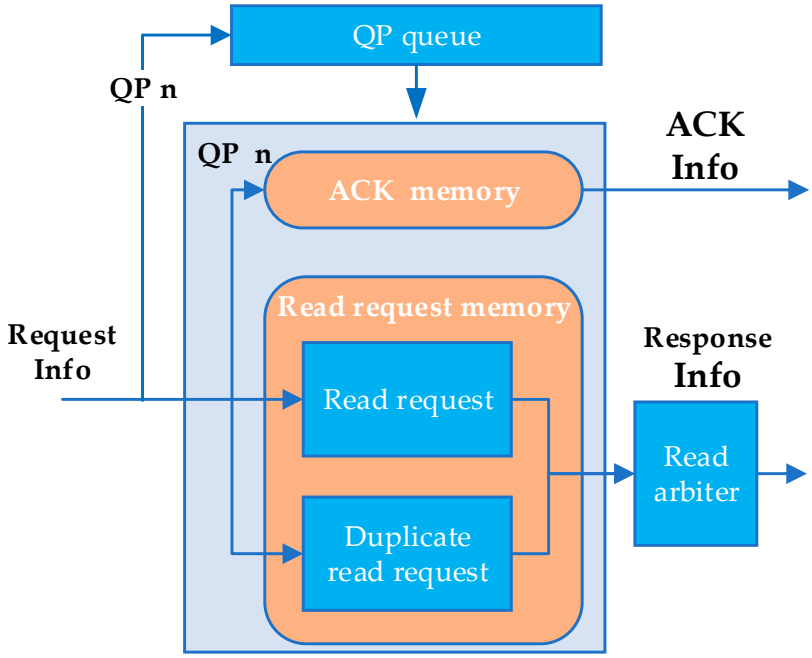

Figure 5. Request manager flow chart.

The preceding RX engine parses the content of the packets, generating two types of request information. First, for write- and send-type packets, an acknowledgement response operation is triggered. Second, for read-type packets, they are refined into read requests or duplicate read requests based on their packet sequence number (PSN) and status information. The request manager subsystem receives and stores these two types of information inputs and then performs subsequent processing scheduling.

After receiving the input information, the QP ID of the request information is first registered with the QP queue module. Depending on the type of information, it is stored in the corresponding storage structures. It includes the ACK memory module, the read request storage module, and the duplicate read request storage module. These three storage modules allocate independent storage space for each QP, with no association between the storage contents of different QPs. The ACK storage module uses an overwriting storage method. Specifically, when the same QP ID receives an ACK message again, it can be directly overwritten into the same entry to complete the accumulation of ACK information. The read request memory module and the duplicate read request storage module sequentially store the corresponding information into RAM.

All Queue Pairs (QPs) have read/duplicate read request memory and ACK memory of the same size, which is also configurable through parameters. By default, each QP has storage space for 16 read/duplicate read request messages, and due to the overwritable nature of ACK information, we only store 1 ACK message.

When subsequent modules are ready, this subsystem begins to schedule arbitration. First, the QP queue module determines the QP ID that was registered the earliest and schedules the response packets that this QP needs to generate. If the QP needs to send a read response packet, the read request arbitration module compares the PSN of the read request and duplicate read request in order. The method schedules read responses with smaller PSNs. The above steps are executed in a loop to complete all pending request information for all QPs.

2.3. QP Manager

QP (Queue Pair) is the fundamental unit of communication in RDMA technology, which consists of a send queue and a receive queue. All work requests are submitted by the host, and the instructions of the WQEs are placed in the appropriate work queue. The

RDMA network card executes the WQEs in the order they are placed in the work queue. When the channel adapter completes a WQE, a completion queue element (CQE) is placed in a completion queue. Each CQE contains all the necessary information to report the completion of the corresponding work to the host.

In our design, the QP manager subsystem is used to assist in the implementation of parser and generator functions. It is responsible for managing all QPs, obtaining configuration information through interaction with the host, and completing the issuance of WQEs and the submission of CQEs. It can be roughly divided into three parts: the QP information module, the WQE acquisition module, and the CQE generation module.

QP information is bound to the QP ID, including base address information for interaction with the remote QP, PSN, remote key, and status information for each local QP. This information is obtained from the host through the AXI lite interface and is stored in this module. We have chosen to use on-chip RAM to store QP registration information and the running status record information of the QP. This saves interaction costs with the host or memory and speeds up the processing speed.

Other subsystem modules in the design read this information and update it according to the operating conditions for verifying and assembling data packets.

WQEs are obtained through the AXI memory map from memory. This module polls each QP and transmits the obtained WQE to the TX engine. The TX engine assembles data packets based on this and executes work requests. The generation of CQEs is triggered by the RX engine; if the corresponding QP's WQE has completed the reception of all data packets, the CQE is written into the corresponding area of memory through the AXI memory map to notify the host that the work is completed.

2.4. TX Engine

As shown in Figure 6, the TX engine obtains the response and request to be executed from the request handler and QP manager subsystems, respectively. It uses the AXI memory-mapped method to fetch the payload for transmission from memory, and after assembly, it carries the output of the second-layer Ethernet packets on the AXI stream bus. The process of packet generation is described in modules as follows.

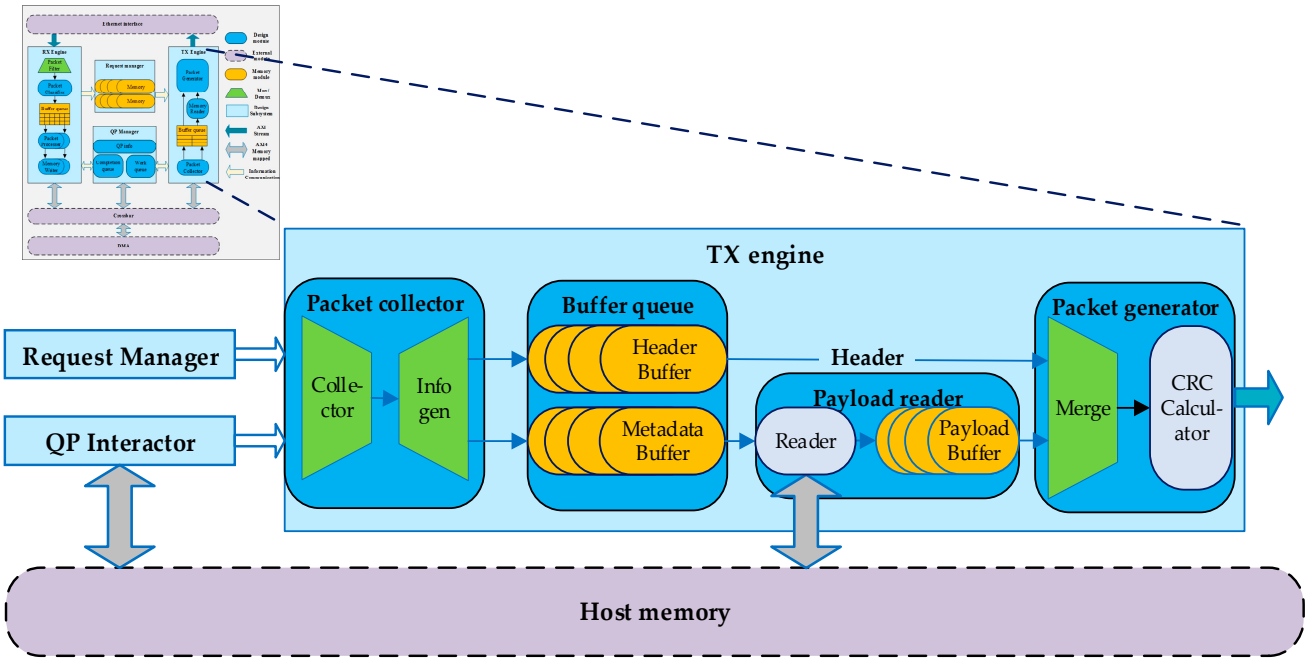

Figure 6. TX engine.

2.4.1. Packet Collector

This module consists of two core components: the send collector module and the information generator module.

The packet collector module collects all the data packets that need to be generated. The request manager needs to generate response packets, while the QP manager needs to generate request packets. When both types are present simultaneously, the response packets are given higher priority than the request packets. This mechanism reduces the completion delay of remote input requests.

The information generator module further specifies the received requirements into two key parts: the packet header and metadata. The packet header is the fundamental component that constitutes the packet and will be used in subsequent modules for packet assembly. In fact, it is based on the collected information to assemble the BTH+ data packet header and determine the transport layer work of the RDMA data packet. At the same time, the UDP/IP/Ethernet layer data are set according to the RDMA connection information and the packet length. Metadata correspond one to one with packet headers to obtain payload. It provides detailed information such as payload length, memory access address, and QP ID. Similar to packet processors in the RX engine, the main time overhead of information generators is also smaller than the memory access time of the subsequent modules. Therefore, sequentially generating information for each data packet does not affect the throughput of the data packet.

These two parts of the content are stored separately in their respective cache queues for subsequent module call processing.

2.4.2. Buffer Queue

The TX engine utilizes two dedicated buffer queues: the packet header buffer and the metadata buffer. The queues are designed in parallel and can be processed and transformed individually. The design depth of these two queues is consistent, and the content maintains a one-to-one correspondence and is consumed simultaneously, ensuring the coupling of a packet's header and payload.

The main task of the packet header cache queue is to directly provide the packet header information required by the packet generator module. Considering the significant differences in packet header lengths under different network layer protocols (IPv4/IPv6) and operations, the queue width is set to a larger value.

Due to the correlation between metadata and payload information, the buffer queue width is determined. It serves the memory read module, which is responsible for providing the necessary information to retrieve and extract the payload of the packet. Once the payload is successfully read, these data are used for subsequent packet assembly work.

Similar to the RX engine's queue, the depth of these two queues is also parameterizable. In the subsequent implementation, the default value is 512.

2.4.3. Payload Reader

The memory reader is responsible for extracting packet payloads from the host memory and caching them. The module is connected to an external DMA function module through a set of AXI memory-mapped interfaces. It utilizes the AXI read address and read data channels, while leaving the write address, write data, and write response channels idle. Specifically, this module organizes the actions of the read address and read data channels based on input information.

Firstly, it receives information from the metadata queue, which contains key parameters required for the read operation, such as the length and starting address of the packet payload. By using these parameters, the memory read module determines the actions of the read address channel.

Once the subsequent reader module is ready, it begins to obtain the packet payload from the specified memory address through the AXI read data channel. The starting address and payload length are both obtained from metadata, thereby converting metadata into the actual payload.

After the read operation is completed, the data are temporarily stored in the payload buffer queue. This buffer queue replaces the previous metadata cache queue, with elements corresponding to the packet header queue. In this way, the subsequent packet generator module can synchronously retrieve the headers and payload for further assembly and processing.

2.4.4. Packet Generator

The packet generator module is responsible for merging scattered packet headers and payloads into complete packets and outputting them in AXI stream format. This module consists of two main components: the packet merger and the CRC calculator.

When both the packet header buffer queue and the payload buffer queue are ready, the merger begins its operation. It retrieves data from both queues. Based on the packet type and the length information recorded in the header, the merger precisely splices the header and payload together to form a packet structure.

Due to the different payload and packet header lengths corresponding to packet length and packet type, the assembly method also needs to be distinguished and processed. Specifically, we have recorded this information in the metadata generated earlier and passed it along with the payload buffer queue to the packet generator. We use this information to assemble various data packets separately.

Subsequently, the CRC calculator module comes into play, calculating the ICRC (Internal Cyclic Redundancy Check) value for the packet. This step ensures the integrity and reliability of the packet during transmission.

The calculated ICRC value is then embedded into the packet, marking the final completion of the packet assembly. The assembled packet is then output to the external Ethernet interface, ready for network transmission.

2.5. Other Modules

In addition to the aforementioned core design modules, to fully implement the RDMA transmission and reception functions, we also need some auxiliary modules to work in concert. These modules include the crossbar module, the Ethernet module, and the DMA module, which together form the supplementary part of the system architecture.

The crossbar module serves to connect all the AXI memory-mapped master interfaces within the design to the target slave interface (DMA module). Each input port shares the address and conducts its own read and write data transfers. This module distinguishes between read and write transactions from the inputs and processes them in parallel. As shown in Figure 7, we take a write transaction as an example. All the write address channels converge into an arbiter, which grants access rights to one input port at a time and places the arbitration results into a queue. The port at the head of the queue writes the data to the target address, completing the transaction. The read operation is similar, executing their respective port read operations based on the read address arbitration results.

The DMA (Direct Memory Access) module is responsible for all accesses to the host memory. Its core function is to convert AXI memory-mapped transactions into messages at the PCIe (Peripheral Component Interconnect Express) transport layer, accessing the corresponding memory areas through the PCIe interface. This module can be implemented by using existing DMA IP cores or open-source projects.

The choice of the Ethernet module depends on the specific network interface requirements. In our design, considering the need to support a transmission rate of 100 Gbps, we have chosen Xilinx Ethernet IP as the network interface.

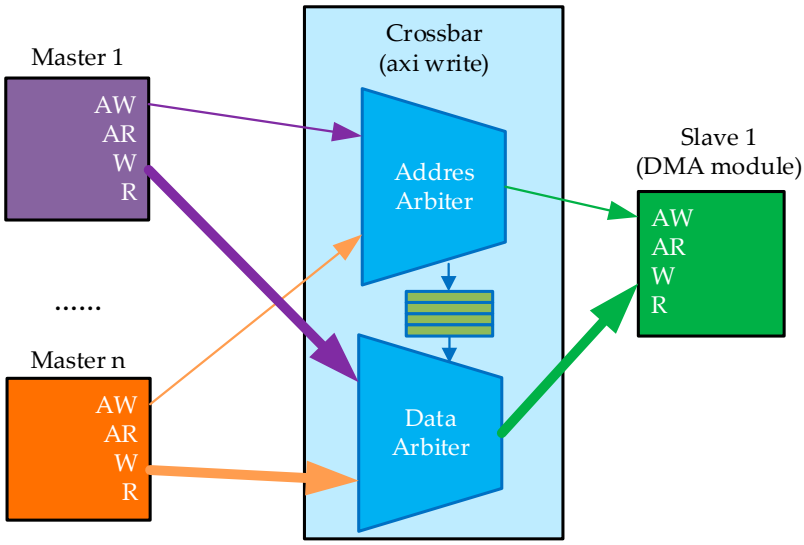

Figure 7. Crossbar (write).

3. Implementation

3.1. Hardware Platform

This design was implemented on the XILINX Alveo U200 accelerator card [11], which was equipped with a high-performance UltraScale + XCU200 FPGA. This FPGA supports a generation 3 × 16-lanes PCIe connector, as well as a QSFP 28 network interface, both of which can meet the high-speed data transmission test requirements of 100 Gbps.

The test server used was the Dell R740XD, which was equipped with an Intel Xeon Silver 4216 processor and ran the Ubuntu 20.04.6 LTS operating system. This configuration provides a stable and efficient computing environment for testing. The testing environment is shown in Figure 8.

Figure 8. Test environment.

To ensure the accuracy and reliability of the test, two Alveo U200 accelerator cards were directly connected via optical fiber, forming a closed-loop test network.

We relied on a high-performance hardware network card to implement the RDMA packet parser and generator functions, which are responsible for handling data and control interactions between the host memory (via PCIe) and the Ethernet.

The Corundum [12], designed by Alex Forencich, is an open-source, high-performance FPGA network card that supports up to 100 Gbps line-rate transmission. Due to its high performance and flexibility, it is suitable for developing and testing new network applications. Therefore, we integrated the design into Corundum for subsequent testing. On the host side, we adapted the AXI memory-mapped and AXI Lite interface functions of our design to the host interface of the Corundum framework. Through Corundum's DMA capabilities, we established a connection with the Xilinx PCIe hard IP [13].

On the network side, the Ethernet interface in our design is connected to Xilinx Ethernet IP [14]. When receiving packets, non-ROCE v2 traffic is diverted by the receive engine and input into the raw Ethernet interface of the Corundum framework. When sending packets, we merge non-ROCE traffic with the output of the transmit engine, forming a network card that integrates RDMA transmission and reception capabilities.

Apart from the instantiation of the IP cores mentioned earlier, we have extensively utilized Xilinx Parameterized IPs in our design. We have used these IPs to construct detailed modules for implementing the RTL modules discussed in Section 2. For instance, we have used XPM_FIFO to store data of the input and output modules. Moreover, we have used XPM_MEMORY as the actual memory for various queues.

Moreover, we have improved the design of the pipeline, so the AXI bus can work on 250 MHz and the throughput performance can reach 100 Gbps.

After the overall project implementation, the resource usage of the TX engine and RX engine we designed is shown in Table 1.

Table 1. Resource usage.

Name	LUT	Registers	CARRY8	F7 Muxes	F8 Muxes	F9 Muxes	Block RAM	URAM
FPGA total	1,182,240	2,364,480	147,780	591,120	147,780	147,780	2160	960
TX engine	12,991	9362	101	54	2	0	17	16
RX engine	17,770	13,796	494	282	66	0	48.5	0

3.2. Software Platform

The software platform we designed consists of two main components: the driver and the user program.

The driver serves as the direct interface with the hardware, relying on the implementation of the corundum driver. This driver connects to the Linux kernel to initialize the network card, register memory interfaces, and allocate DMA access buffers, etc. Additionally, it uses the register space to pass network transmission parameters to the hardware. We have extended the driver to support the issuance of RDMA configuration parameters and the interaction of key information in transaction transactions. This allows RDMA data to access host memory through DMA.

The user program is run on top of the driver, focusing on the control and management of RDMA transactions. Its operations include registering and allocating memory space, creating Queue Pairs, and establishing RDMA connections with remote parties. It also involves generating various transaction work requests and sending them to the corresponding Queue Pairs. Furthermore, it manages application transactions and monitors the completion status of these operations.

3.3. Performance Evaluation

In actual RDMA transfer, the one-sided operations of WRITE/READ offer greater advantages. These operations do not require the involvement of the responding side's CPU, unlike the two-sided operations of SEND/RECV. Therefore, SEND/RECV is usually only used to transmit some control information, while RDMA WRITE/READ is the type of operation applied when transferring a large amount of data.

Therefore, we used the previously mentioned test environment to test the RDMA READ and RDMA WRITE throughput performance. The two FPGA cards acted as the client and the server. Requests were initiated and completed by the client and responded to by the server. The read operation payload was retrieved from the server to the client. When the last read response packet was received, the operation was completed, and a CQE was generated. The write operation transmitted the payload from the client to the server and generated the CQE when the last acknowledgement was received.

We wrote a test program for both the server and the client to measure the throughput that this design can achieve under different sizes of WQEs. We measured the total time from

when the client issues a work request to when it listens and receives the completion queue, calculating the transfer speed under different loads with the same Maximum Transmission Unit (MTU). The calculation of throughput is shown in Formula (2) below.

$$\text{throughput} = \frac{\text{message size}}{t_{\text{completion}} - t_{\text{work request}}} \tag{2}$$

As shown in Figure 9, we can observe that the performance of the read and write operations increases with the growth of the message size in work requests. The reason is that for a smaller size, the time required to establish a connection takes a larger proportion of the entire transmission transaction time, so the throughput performance is low. As the size increases, throughput performance improves. When the message size reaches 4096B or larger, RDMA can use the maximum transmission unit. When the message size is sufficiently large, the throughput performance is close to 100 Gbps.

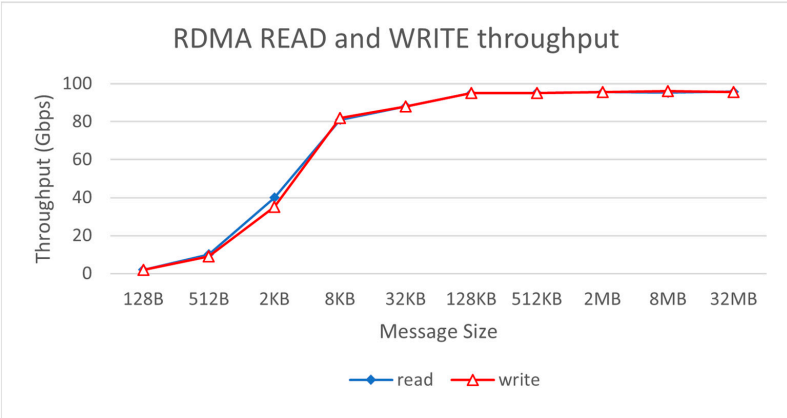

Figure 9. RDMA throughput.

To test the performance of this network card, we deployed the Mellanox Connectx-5 [15] network card in the same environment for comparison. Perftest [16] is a test suite maintained by the Open Fabrics Alliance organization, which can be used to evaluate the performance of RDMA network cards. We used perftest to test the read–write performance of our design and the Mellanox CX5 network card. The results obtained are shown in Figure 10, and the peak performance of the two is basically equivalent. Moreover, compared with ASIC-based dedicated network cards, our implementation on an FPGA has greater flexibility.

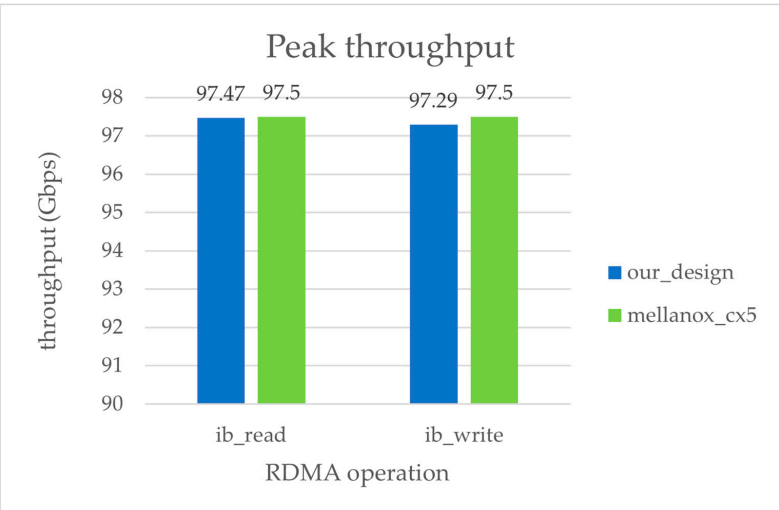

Figure 10. Throughput performance results.

4. Conclusions

In this paper, we propose an implementation scheme for receiving and transmitting RDMA data packets on an FPGA-based smart network interface card. This scheme supports reliable connection-oriented RDMA operations and completes the offloading of the RoCE v2 protocol. We tested our design on a "XILINX Alveo U200 data center accelerator card" with a maximum throughput of close to 100 Gbps. Moreover, we compared our design with a dedicated RDMA network card (Mellanox CX5) in the same test environment, and the RDMA READ and RDMA WRITE throughput performance was generally consistent, reaching the maximum throughput in that environment. In addition, the design presents standard bus interfaces in the overall architecture, thus offering strong scalability and providing the possibility of adding intelligent network card functions such as forwarding and scheduling on the transmission and reception ends, enhancing flexibility and practicality in practical applications.

The method of RDMA packet parsing and packet generator proposed in this paper is innovative and practical and has a wide application prospect.

5. Future Work

The contents of the whole RDMA system are very broad. However, the focus of this paper is the RDMA packet parser and packet generator. These are the basic contents of RDMA. In future work, we will implement the selective retransmitting method and the out-of-order packet handling method.

Selective retransmission can be achieved by extending packet loss recording and localization modules beyond the RX engine and TX engine. In addition, for congestion control issues, the ECN can be used to achieve packet generation speed by sending and receiving CNP packets. Then, for the scheduling aspect of multitasking and multi QPs, various scheduling strategies can also be extended based on the design in this article.

In summary, the design mentioned in this article can serve as the foundation for multiple tasks and expand more RDMA functionalities. We believe that other developers may draw inspiration from this architecture for expansion and develop further innovations.

Author Contributions: Conceptualization, Z.G.; methodology, implementation, Z.S.; integration, J.M.; validation, Y.P.; writing—original draft preparation, Z.S.; writing—review and editing, Z.G. and Z.S.; supervision, Z.G. and Z.S. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by Oriented Project Independently Deployed by Institute of Acoustics, Chinese Academy of Sciences: Research and Development of Key Technologies and Equipment for Low Latency Interconnection Network in Intelligent Computing Center Cluster (Project No. MBDK202401).

Data Availability Statement: All the necessary data are included in the article.

Acknowledgments: The author would like to thank Mangu Song for her insightful comments.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. InfiniBand Trade Association. InfiniBand Architecture Specification Release 1.4 Annex A17: RoCEv2. 2020. Available online: https://www.infinibandta.org/ (accessed on 10 August 2024).
2. Technologies, M. RoCE in the Data Center. 2014. Available online: https://network.nvidia.com/related-docs/whitepapers/roce_in_the_data_center.pdf (accessed on 10 August 2024).
3. Bai, W.; Abdeen, S.S.; Agrawal, A.; Attre, K.K.; Bahl, P.; Bhagat, A.; Bhaskara, G.; Brokhman, T.; Cao, L.; Cheema, A. Empowering azure storage with {RDMA}. In Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), Boston, MA, USA, 17–19 April 2023; pp. 49–67.
4. Leung, L.; Kim, J. OCI accelerates HPC, AI, and database using RoCE and NVIDIA ConnectX. 2023. Available online: https://blogs.oracle.com/cloud-infrastructure/post/oci-accelerates-hpc-ai-db-roce-nvidia-connectx (accessed on 24 July 2023).
5. Sha, M.; Guo, Z.; Song, M. A Review of FPGA's Application in High-speed Network Processing. *J. Netw. New Media* **2021**, *10*, 1–11.

6. Firestone, D.; Putnam, A.; Mundkur, S.; Chiou, D.; Dabagh, A.; Andrewartha, M.; Angepat, H.; Bhanu, V.; Caulfield, A.; Chung, E. Azure accelerated networking:{SmartNICs} in the public cloud. In Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), Renton, WA, USA, 9–11 April 2018; pp. 51–66.
7. Krishnan, V.; Serres, O.; Blocksome, M. COnfigurable Network Protocol Accelerator (COPA): An Integrated Networking/Accelerator Hardware/Software Framework. In Proceedings of the 2020 IEEE Symposium on High-Performance Interconnects (HOTI), Piscataway, NJ, USA, 19–21 August 2020; pp. 17–24.
8. Korolija, D.; Roscoe, T.; Alonso, G. Do {OS} abstractions make sense on {FPGAs}? In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), Online, 4–6 November 2020; pp. 991–1010.
9. Arashloo, M.T.; Lavrov, A.; Ghobadi, M.; Rexford, J.; Walker, D.; Wentzlaff, D. Enabling Programmable Transport Protocols in High-Speed NICs. In Proceedings of the Networked Systems Design and Implementation, Santa Clara, CA, USA, 25–27 February 2020.
10. InfiniBand Trade Association. InfiniBand Architecture Specification Release 1.4 Chapter 7 Section 8 CRCS. 2020. Available online: https://www.infinibandta.org/ (accessed on 10 August 2024).
11. Devices, A.M. Alveo U200 and U250 Accelerator Cards User Guide (UG1289). 2023. Available online: https://docs.amd.com/r/en-US/ug1289-u200-u250-reconfig-accel/Please-Read-Important-Legal-Notices (accessed on 10 August 2024).
12. Forencich, A.; Snoeren, A.C.; Porter, G.; Papen, G. Corundum: An open-source 100-gbps nic. In Proceedings of the 2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Fayetteville, AR, USA, 3–6 May 2020; pp. 38–46.
13. Devices, A.M. UltraScale+ Devices Integrated Block for PCI Express Product Guide. 2024. Available online: https://docs.amd.com/r/en-US/pg213-pcie4-ultrascale-plus (accessed on 10 August 2024).
14. Devices, A.M. UltraScale Devices Integrated 100G Ethernet Subsystem Product Guide. 2024. Available online: https://docs.amd.com/r/en-US/pg165-cmac (accessed on 10 August 2024).
15. Technologies, M. Connectx-5-Datasheet. Mellanox ConnectX-5 Product Brief 2020. Available online: https://network.nvidia.com/files/doc-2020/pb-connectx-5-en-card.pdf (accessed on 10 August 2024).
16. OFED. Perftest. 2024. Available online: https://github.com/linux-rdma/perftest?tab=readme-ov-file (accessed on 10 August 2024).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.